LLM-Assisted Security Auditing and Fuzzing of libxml2: Hypothesis Generation with Evidence-Driven Triage

Abstract

libxml2 is a mature and widely deployed XML parsing library, yet some of its public APIs and stateful code paths are rarely reached by existing fuzz targets. Large language models (LLMs) can widen exploration by proposing bug hypotheses and drafting test harnesses, but their output is unreliable: generated harnesses misuse APIs, and suspicious code is not automatically a vulnerability. This paper describes a multi-engine framework that combines (1) LLM-assisted source auditing, (2) differential and metamorphic testing, and (3) harness generation with coverage-guided fuzzing, all feeding a shared triage pipeline that accepts a finding only on deterministic evidence. Applied to seven libxml2 subsystems, the audit prioritized one primary lead, a repeated integer size-arithmetic pattern in the regex/automata engine (`regexp.c`) that spans five functions, and documented two further candidates: catalog recursion-depth state handling and an XML-save indentation progress loop. The experimental record contains 72 differential runs with no disagreements and no accepted fuzzing crash; one earlier crash candidate was rejected as harness misuse. We report these as candidates under validation, state exactly what evidence is still missing, and give a concrete validation plan. The main contribution is a methodology that uses the LLM to generate hypotheses and uses reproducible testing to accept or reject them.

Keywords: fuzzing, large language models, libxml2, vulnerability discovery, differential testing, crash triage

1. Introduction

Parsers for untrusted input are a long-standing source of memory-safety and availability bugs. libxml2 [1] is embedded in a large number of applications and handles hostile input routinely, so defects in it have wide impact. The library has been fuzzed extensively, for example by continuous services such as OSS-Fuzz [2], but coverage-guided fuzzing is only as good as its harnesses. A byte-oriented harness exercises the parts of the API it was written for, and APIs that need structured objects, multi-step setup, or shared state are harder to reach.

LLMs offer two things that help here. They can read source code and propose concrete hypotheses about where invariants may break, and they can draft harnesses for entry points that have none. They also introduce three problems:

- **Coverage gaps remain.** A fuzz target may exercise only a subset of the public API and internal state transitions. A review of the repository's top-level harness list found no matching harness for `catalog.c` and `c14n.c`.
- **Harness reliability is low.** Generated harnesses can omit required setup, violate memory-ownership rules, or fail to compile. A crash is meaningful only after the harness has been ruled out as the cause.
- **False positives are common.** A suspicious expression or a sanitizer crash is not a library vulnerability until reachability, input control and reproducibility are shown.

We therefore adopt one research principle: *use the LLM to widen exploration and formulate hypotheses; use deterministic testing, source tracing and independent reproduction to accept or reject them.*

1.1 Contributions

1. A three-engine framework (audit, differential testing, harness-based fuzzing) with a shared, evidence-based triage pipeline.
2. A coverage-gap assessment that turns a harness-versus-module comparison into a prioritization signal, with its limits stated.
3. A prioritized candidate set from the audit: one primary memory-safety lead and two supporting logic/availability leads, each with a trigger model and an explicit status.
4. A record of negative results, meaning hypotheses rejected by traced invariants, and an honest account of what the experiments do and do not show.
5. A validation plan that specifies the evidence needed before any candidate is reported to maintainers.

2. Background and Related Work

Coverage-guided fuzzing. Fuzzing began with random-input testing of UNIX utilities [3]. Mutation-based fuzzers such as AFL-family tools [3] , [3] and libFuzzer [3] use coverage feedback to evolve inputs [3] , and are usually paired with sanitizers (AddressSanitizer [4] , UndefinedBehaviorSanitizer [4]) to expose memory errors and undefined behavior. Their effectiveness depends strongly on harness quality and seed corpora.

Differential and metamorphic testing. Comparing equivalent inputs across parsing paths, or checking that semantics-preserving transformations leave results unchanged [5] , [5] , can reveal inconsistencies that never crash and so are invisible to crash-only fuzzing.

LLMs for security testing. Recent work uses LLMs to write fuzz drivers [6] , [6] , [6] , suggest seeds and review code for defects. Reported weaknesses include invalid drivers, hallucinated API usage and unsupported vulnerability claims. This motivates our requirement that no LLM suggestion is accepted without deterministic evidence.

3. System Design and Methodology

The framework selects testing targets and routes them to three engines. Their results go to one triage pipeline (Figure 1 in the final paper; described textually here).

3.1 Target selection

Targets come from four sources: public APIs that existing fuzzers exercise poorly (coverage gaps), historically fixed bugs that suggest similar weaknesses, recently modified code, and static-analysis warnings. The in-scope subsystems for this study were catalog resolution (`catalog.c`), canonicalization (`c14n.c`), XML serialization (`xmlsave.c`), the regex/automata engine (`regexp.c`), XPointer (`xpointer.c`), pattern matching (`pattern.c`) and XML Schema datatypes (`xmlschemastypes.c`).

The supplied review compared the harnesses in `fuzz/` with the source modules. Harnesses exist for `xml.c` , `html.c` , `reader.c` , `regexp.c` , `schema.c` , `uri.c` , `xinclude.c` and `xpath.c` , while `catalog.c` and `c14n.c` have no matching top-level harness. These two were selected for manual review because their API shape and object/state requirements may be less directly exercised by byte-oriented targets.

> **Limitation.** The absence of a one-to-one harness does not prove zero coverage. A module may be reached indirectly by another target, by tests or by downstream callers. This list is a prioritization signal, not a coverage measurement.

3.2 Engine 1: LLM-assisted code audit

The model reads selected source and produces concrete hypotheses. The auditor traces control flow and data flow, compares sibling branches, and *rejects* a hypothesis when bounds or invariants prove it safe. Suspicions are prioritized, and where possible a candidate proof-of-concept is compiled and run under sanitizers.

3.3 Engine 2: Differential and metamorphic testing

The engine generates XML inputs and compares alternative parsing or transformation paths. The recorded test categories were entities, namespaces, encodings, internal DTD subsets, character references, and CDATA/comments/processing instructions, with

twelve inputs per category. A disagreement is treated as a candidate that must be minimized and given a semantic explanation.

3.4 Engine 3: Harness generation and coverage-guided fuzzing

The engine generates or adapts C/C++ harnesses for uncovered entry points, checks them by compilation and execution, and runs libFuzzer with sanitizers where available. Coverage and crash artifacts are collected for triage.

3.5 Shared triage

Every result passes through the same six steps:

1. Reproduce the result.
2. Minimize the input.
3. Check sanitizer and stack evidence.
4. Verify that the fault is inside libxml2 and not in the harness.
5. Deduplicate.
6. Assess reachability and impact.

4. Findings

All findings below come from source tracing and have not yet been reproduced dynamically. We describe each with its trigger model and status.

4.1 Primary lead: regex size arithmetic (`regexp.c`)

A token-concatenation pattern repeats across five functions: `xmlRegExecPushString2` , `xmlAutomataNewTransition2` , `xmlAutomataNewNegTrans` , `xmlAutomataNewCountTrans2` and `xmlAutomataNewOnceTrans2` . Two string lengths are stored in `int` variables (`lenn` and `lenp`). The expression `lenn + lenp + 2` decides whether the function allocates heap storage or uses a fixed 150-byte stack buffer, and the following `memcpy` calls still copy the original lengths.

If the size expression is evaluated incorrectly for very large inputs and selects a destination that is too small, the copies can exceed its bounds. The stack-buffer variant is in `xmlRegExecPushString2` , and the heap-allocation variants share the same size computation. Because the pattern repeats, it is a shared code-pattern audit target rather than an isolated line.

- **Potential impact:** stack or heap out-of-bounds write if the computation selects undersized storage.

- **Reachability:** direct API use with extremely large, attacker-influenced strings. A path through ordinary parsed XML has not been established.
- **Caveat:** signed overflow is undefined behavior in C, so the actual branch outcome depends on size conversion and compiler semantics and must be demonstrated on the target build before any working overflow is claimed.
- **Status:** high-priority memory-safety candidate; requires target-build reproduction.

4.2 Candidate: catalog recursion state (`catalog.c`)

In `xmlCatalogXMLResolve()`, on the `haveNext` resolution path, a recursive-resolution failure followed by a depth-limit check can return `NULL` without executing the `catal->depth` decrement used by the neighboring success and fall-through paths. The review traced a possible self-referential `nextCatalog` chain in which catalog caching lets recursive calls revisit the same catalog entry. If the shared depth field is not restored on unwinding, later lookups may keep seeing the depth limit as exceeded.

- **Trigger model:** a cyclic `nextCatalog` reference plus cache reuse. The route from an attacker-controlled processing instruction into this path still needs end-to-end verification.
- **Potential impact:** persistent catalog-resolution failure. No memory corruption is alleged.
- **Status:** logic/availability candidate; a focused regression test should confirm the state-poisoning behavior before severity is set.

4.3 Candidate: XML save progress loop (`xmlsave.c`)

On the path `xmlSaveSetIndentString()` → `xmlOutputBufferWriteWSNonSig()`, the failure path for an empty or invalid indentation string can return before assigning safe values to `indent_size` and `indent_nr`, leaving a zero increment. The output loop advances its index by `ctxt->indent_nr`. If that value is zero while the loop condition stays true for nested content, the index never advances and serialization can hang.

- **Trigger model:** application-side use of an empty or invalid global indentation setting, followed by serialization with `XML_SAVE_WSNONSIG`. Ordinary untrusted XML content alone was not shown to set this option.
- **Potential impact:** CPU-consuming serialization hang.
- **Status:** configuration-dependent robustness candidate, lower priority than the regex lead.

4.4 Severity summary

Candidate	Module	Class	Reachability	Status
Regex size arithmetic	`regexp.c`	Possible out-of-bounds write	Direct API, very large strings; XML path not shown	Needs target-build reproduction
Catalog recursion state	`catalog.c`	Logic / availability	Via cyclic cached resolution; document route unverified	Logic candidate
XML save progress loop	`xmlsave.c`	Robustness / availability	Needs invalid application-side configuration	Configuration-dependent

No remote-code-execution claim is made. The regex candidate ranks first because allocation-size arithmetic is followed by memory-copy operations; its exploitability depends on reproducible behavior and a realistic input path.

5. Experimental Results

5.1 Recorded activity

Measure	Result
Candidate issue patterns documented	3
Regex/automata functions sharing the size pattern	5
Differential runs (6 categories × 12 inputs)	72
Differential disagreements	0
Accepted fuzzing crashes in the recorded run set	0

In an earlier evaluation (25–26 October 2026, revision `c79c9612fcc77b06b0d16f35fd363d938bdbbf5e`), a generated harness for `xmlBufGetNodeContent` produced a reproducible heap-use-after-free report that was rejected during triage because the harness appeared to violate memory-ownership rules. A later six-chunk (900 s each) coverage-guided run on the `reference_domMutation` harness reported no crashes. The logged coverage counts there are harness-reported counters, not a source-coverage percentage, and are not used as evidence in this paper.

These results show that the workflow runs end to end. The absence of a crash is not evidence that the library is bug-free.

5.2 Crash artifact validation

One crash artifact, named `1793706967537_crash`, was reviewed. It was reported as two bytes (`0x05 0x05`) and was not accompanied by a sanitizer summary, stack trace, fault address, signal details or a complete reproducer. It therefore cannot be attributed to libxml2 or linked to the regex candidate. It is kept as a triage lead and is **not** counted as a validated vulnerability.

The evidence still required is: a complete reproducer, the exact source revision and build flags, an ASan/UBSan report, the faulting stack trace, the allocation size and copy length, and confirmation that the failure occurs inside the library rather than the harness.

5.3 Negative results

Besides candidates, the audit recorded hypotheses that were rejected by traced invariants:

- **`c14n.c`:** namespace-stack growth, allocation and reallocation behavior, namespace rendering and node-set handling. No concrete memory-safety violation was established.
- **`xpointer.c` and `pattern.c`:** XPointer buffer sizing and escaped-character consumption; pattern compiler error-path cleanup and sentinel-dependent lookahead. The hypotheses were not supported by the invariants.
- **`xmlschemastypes.c`:** date/duration arithmetic, bounded month indexing, integer parsing buffers and decimal formatting bounds. No overflow or out-of-bounds write was constructed.
- **Catalog URN unescaping:** the suspected `urn[1]` / `urn[2]` lookahead was ruled out because short-circuit evaluation and the string terminator prevent the out-of-bounds read.

Recording these results matters: it shows the LLM's suggestions were filtered rather than accepted, and it saves later reviewers from repeating the same checks.

6. Discussion

What worked. The audit narrowed a broad search to specific, testable code paths. The repeated regex size pattern is the kind of cross-function finding that a single-function review can miss, and the triage rules prevented a harness-caused crash from being reported as a library bug.

What did not. Generated harnesses did not always compile or reach their intended targets, model-provider rate limits interrupted parts of the LLM workflow, and a plateaued harness was not reliably turned into a meaningfully different test strategy.

Why candidates are not findings. Each candidate rests on source tracing alone. The regex lead depends on compiler and architecture behavior; the catalog lead depends on a document-to-catalog route that is unverified; the save-loop lead depends on application configuration. Calling any of them a vulnerability now would be unsupported.

7. Threats to Validity

- **Internal:** source tracing can be wrong. Without runtime evidence, a candidate may be unreachable or prevented by a check outside the traced path.
- **Construct:** the coverage-gap list is a prioritization heuristic, not coverage. Harness-reported coverage counters cannot be read as a percentage of the library.
- **External:** results come from one library, seven subsystems and one campaign. They do not support claims about LLM-assisted fuzzing in general.
- **Reproducibility:** the exact commit, compiler and build flags for the candidate analysis must be fixed and published with the final paper.

8. Validation Plan

1. Pin the libxml2 commit and the compiler and build flags.
2. Reproduce the regex size calculation on the target architecture.
3. Add boundary tests around `int` / `size_t` conversion and allocation selection.
4. Run ASan and UBSan and keep the full logs.
5. Test catalog recursion-state restoration and XML-save loop progress.
6. Minimize each reproducer and verify it independently.
7. Measure source coverage with an LLVM coverage report on the exact instrumented build, stating the metric, its denominator and its scope.
8. Compare against established libxml2 harnesses using the same flags, sanitizers and time budgets.

For the regex lead, a suggested defensive fix is to use `size_t` for lengths and check the addition before allocation (for example, reject if `lenp > SIZE_MAX - lenn - 2`), and to verify the actual `memcpy` bounds instead of relying on signed-overflow wraparound.

9. Conclusion

We presented a framework that pairs LLM exploration with deterministic, evidence-driven triage, and applied it to libxml2. The audit prioritized a repeated regex size-arithmetic pattern across five functions, traced two further state and progress candidates, and documented why several other hypotheses were rejected. The experiments recorded 72 differential runs with no disagreements and no accepted fuzzing crash. No candidate is yet a confirmed vulnerability.

The next milestone is a minimal, sanitizer-backed reproducer for the regex lead, followed by validation of the catalog and XML-save candidates, and submission to maintainers only of reports that the evidence supports. We argue the framework should be judged on independently validated findings and reproducible coverage gains, not on the number of LLM suspicions or raw crashes.

References

1. GNOME Project, "libxml2: The XML C parser and toolkit." [Online]. Available: https://gitlab.gnome.org/GNOME/libxml2
2. K. Serebryany, "OSS-Fuzz: Google·s continuous fuzzing service for open source software," invited talk, USENIX Security Symposium, 2017.
3. M. Zalewski, "American Fuzzy Lop (AFL)"; A. Fioraldi et al., "AFL++: Combining incremental steps of fuzzing research," WOOT, 2020; LLVM Project, "libFuzzer."
4. K. Serebryany et al., "AddressSanitizer: A fast address sanity checker," USENIX ATC, 2012; LLVM Project, "UndefinedBehaviorSanitizer." [Online].
5. W. M. McKeeman, "Differential testing for software," Digital Technical Journal, vol. 10, no. 1, pp. 100·107, 1998; T. Y. Chen et al., "Metamorphic testing," Tech. Rep. HKUST-CS98-01, 1998.
6. C. Zhang et al., "How effective are they? Exploring large language model based fuzz driver generation," ISSTA, 2024; Y. Deng et al., "Large language models are zero-shot fuzzers," ISSTA, 2023; C. S. Xia et al., "Fuzz4All: Universal fuzzing with large language models," ICSE, 2024.